MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 7

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

Goal **3**

1 Part 1: fork + exec + wait **3**

2 Part 2: A zombie, on purpose **7**

3 Part 3: Redirection, ls -l > out.txt **9**

4 Part 4: A pipe, ls /etc | wc -l **11**

Conclusion **15**

References **15**

Goal

There are three main questions: how a process is born, where printf writes, and how the command ls | wc -l works. The purpose of this laboratory work is to answer them by running the answers instead of reading them: four short C programs are compiled on Linux with gcc, executed, changed in one line, and the effect of the change is observed in the terminal.

The work has four parts. The first part shows the birth of a process with the three system calls fork, exec and wait, the same three steps that a shell performs for every command that is typed. The second part creates a zombie process on purpose, by delaying the wait of the parent, and observes it in the process table. The third part reproduces the redirection ls -l > out.txt by hand, by closing the descriptor 1 before opening the file, which shows that redirection is done by the shell and not by the program that is redirected. The fourth part builds the pipeline ls /etc | wc -l by hand, with one pipe and two children, and then breaks it on purpose by leaving a writing end open, in order to see why every unused end of a pipe must be closed.

The programs were written and compiled in the directory os-lab7 of the GitHub repository directory with the command gcc -Wall -Wextra, and all the outputs shown in this report were produced on that machine.

Part 1: fork + exec + wait

Questions:

- The line perror("exec") is in the child, right after execlp. Why is it printed only with nosuchprog?
- Which number does the child print as "my parent", and whose PID is it?
- What status does the parent see with ls, and what status with nosuchprog?

Answers:

- A successful execlp does not return: it replaces the program of the calling process, so the instructions that follow it no longer exist in memory and perror is never reached. With ls and with date the call succeeds, therefore nothing is printed. With nosuchprog the program is not found in the directories of PATH, execlp returns -1 and sets errno to ENOENT, so the next line is executed and prints exec: No such file or directory.
- The child prints the number returned by getppid(), which is the PID of the parent, that is, of the process ./run that created it. The parent prints my PID is 14506 and the child prints my parent is 14506, so the two numbers are the same and prove the relation between the two processes.
- With ls the status is 0, because ls finished normally, the same happens with date. With nosuchprog the status is 127, because execlp failed and the child itself ended with exit(127); 127 is the code that the shell also uses for "command not found".

run.c – the program performs the three steps of the shell for the command ls -l. It prints its own PID, calls fork to copy itself, and from that moment two processes execute the same code: fork returns 0 in the child and the PID of the child in the parent, and this value is the only thing that separates them. The child prints its PID and that of its parent and then calls execlp, which replaces its memory image with the program ls; the parent calls waitpid and remains blocked until the child ends, after which it prints the status extracted with WEXITSTATUS. The calls fflush(stdout) are needed because the standard output is buffered, and fork would otherwise copy the buffer into the child and print the same line twice.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                              /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                          /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);                    /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```



Figure 1.1: Compiling and running run.c: the child becomes `ls -l`

gcc -Wall -Wextra -o run run.c && ./run – compiles without any warning and executes the program. The parent is 14506 and the child is 14507, and the child confirms that its parent is 14506. Between the two lines of the program appears the listing produced by ls -l, with the two files run and run.c; the listing is printed by ls and not by run, because the child has already become another program. The last line, printed by the parent after waitpid, shows the status 0. The order of the lines is the proof of the mechanism: the parent writes first, the child and ls write in the middle, and the parent writes the final line only after the child has ended.



Figure 1.2: The modified `execlp` line: the child becomes `date`

execlp("date", "date", (char *)NULL); – only this line of the child was changed. Nothing else in the program knows which command will be executed, which shows that fork and wait do not depend on the program that is launched.

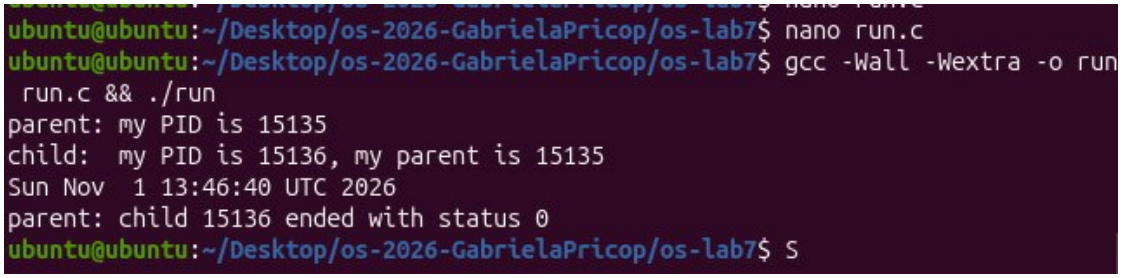


Figure 1.3: The same program with `date` instead of `ls -l`

./run (with date) – the parent is 15135 and the child is 15136, and instead of the listing the program prints the line Sun Nov 1 13:46:40 UTC 2026, which is the output of date. The status is again 0, and perror prints nothing, because the call succeeded.

```
if (pid == 0) {                                    /* only the child runs this */
    printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
    fflush(stdout);
    execlp("nosuchprog", "nosuchprog", (char *)NULL);  /* step 2: become ls>
    perror("exec");                                /* runs only if exec failed */
    exit(127);
```

Figure 1.4: The `execlp` line asking for a program that does not exist

execlp("nosuchprog", "nosuchprog", (char *)NULL); – the name given to execlp is a program that is not installed on the system. The child is created in the same way, but it cannot become anything else and continues to execute the code of run.

```
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab7$ nano run.c
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab7$ gcc -Wall -Wextra -o run
 run.c && ./run
parent: my PID is 15517
child:  my PID is 15518, my parent is 15517
exec: No such file or directory
parent: child 15518 ended with status 127
```

Figure 1.5: The failure of `exec` and the status 127

./run (with nosuchprog) – the parent is 15517 and the child is 15518; the child prints its line, then execlp fails and perror writes exec: No such file or directory on the standard error. The child ends with exit(127) and the parent reports the status 127. The failure happens in the child only: the parent is intact and continues normally, which is exactly the behaviour of a shell that prints "command not found" and then shows its prompt again.

Program of the child	**Parent PID**	**Child PID**	**Status**	**Is perror printed?**
ls -l	14506	14507	0	no, exec succeeded
date	15135	15136	0	no, exec succeeded
nosuchprog	15517	15518	127	yes, ENOENT

Table 1: The three executions of run.c and the status seen by the parent

Table 1 shows that the parent always learns how its child ended, and that the status distinguishes a normal end (0) from a failed exec (127), without the parent knowing anything about the program that was executed.

Part 2: A zombie, on purpose

Questions:

- Which letter does ls have in the STAT column, and what does it mean?
- After 20 seconds, is the [ls] <defunct> line still there? Which line of zombie.c made it disappear?

Answers:

- The process [ls] <defunct> has the letter Z in the STAT column, which means zombie: the child has already finished and has released its memory, but the kernel still keeps its entry in the process table, together with its exit status, because the parent has not read it yet. The parent ./zombie has the letter S, interruptible sleep, because it is inside sleep(20). The word <defunct> and the brackets around the name show that the program no longer exists in memory.
- No. After 20 seconds the line has disappeared: ps shows only the shell and the command ps itself, and the shell reports [1]+ Done ./zombie > /dev/null. The line that made it disappear is waitpid(pid, &status, 0);, which is executed as soon as sleep(20) ends: it reads the status of the child and the kernel then removes the entry from the table. The states are summarized in Table 2.

```
    int status;
    sleep(20); /*new line*/
    waitpid(pid, &status, 0);                    /* step 3: wait for the child >
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Figure 2.1: The line `sleep(20)` added before `waitpid` in zombie.c

zombie.c – the file is a copy of the first version of run.c, the one in which the child becomes ls, with a single line added: sleep(20); placed immediately before waitpid. During these 20 seconds the child has already ended but the parent has not yet called waitpid, and this interval is exactly the life of a zombie. Nothing else was changed, which shows that a zombie is not an error of the program but a normal state between the end of the child and the reading of its status.

```
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab7$ gcc -Wall -Wextra -o zom
bie zombie.c
./zombie > /dev/null &
sleep 2; ps -o pid,ppid,stat,cmd
[1] 16253
    PID    PPID STAT CMD
  13204   13162 Ss   /usr/bin/bash
  16253   13204 S    ./zombie
  16255   16253 Z    [ls] <defunct>
  16257   13204 R+   ps -o pid,ppid,stat,cmd
```

Figure 2.2: The process table while the parent sleeps: `[ls] <defunct>` in state Z

./zombie > /dev/null & ; sleep 2 ; ps -o pid,ppid,stat,cmd – the program is started in the background, so that the shell remains usable, and its output is sent to /dev/null because the listing of ls is not interesting here. The shell confirms the background job with [1] 16253. After two seconds, ps shows the shell /usr/bin/bash with the PID 13204, the parent ./zombie with the PID 16253 in state S, the zombie [ls] <defunct> with the PID 16255 and the parent 16253 in state Z, and the command ps itself with the PID 16257 in state R+. The column PPID of the zombie proves that it is the child created by ./zombie.



Figure 2.3: The process table after the 20 seconds: the zombie has been reaped

ps -o pid,ppid,stat,cmd (after 20 seconds) – the same command run again shows only the shell 13204 and the new ps 16509. Both ./zombie and the zombie child have disappeared: the parent woke up, called waitpid, printed its last line into /dev/null and ended, and the shell announced [1]+ Done. If the parent had ended without waiting, the child would have been adopted by init (PID 1), which calls wait continuously and would have removed it as well.

Moment	**Process**	**STAT**	**Explanation**
after 2 seconds	./zombie (16253)	S	blocked inside sleep(20)
after 2 seconds	[ls] <defunct> (16255)	Z	ended, status not read yet
after 20 seconds	[ls] <defunct>	–	removed by waitpid
after 20 seconds	./zombie	–	ended, [1]+ Done

Table 2: The life of the zombie process, before and after waitpid

Table 2 shows that a zombie occupies no memory but one entry of the process table, and that the only way to remove it is for its parent to read its status with wait or waitpid.

Part 3: Redirection, ls -l > out.txt

Questions:

- Which number did open give, and why exactly that one?
- ls was not changed at all. Why does its listing not appear on the screen?
- To redirect the errors instead (like 2> err.txt), which descriptor would you close before open?

Answers:

- open returned the descriptor 1, as the message open gave descriptor 1 shows. The rule of the kernel is that open always returns the smallest free descriptor of the process. The child inherited the descriptors 0, 1 and 2 from the parent, and close(1) freed exactly the number 1, so it became the smallest free one and it was given to out.txt.
- Because ls writes, as always, into the descriptor 1, without knowing what is behind it. In the child this descriptor no longer points to the terminal but to the file out.txt, and the whole listing goes there; head -3 out.txt reads it back. Redirection is therefore done entirely by the process that calls exec, before the program starts, and the program itself needs no modification. This is the reason why the same mechanism works for every command of the shell.
- The descriptor 2, which is the standard error. Closing 2 and then calling open would make open return 2, since it would be the smallest free number, and all the messages written by perror and fprintf(stderr, ...) would go into the file. The three descriptors are listed in Table 3.

```
 GNU nano 8.7.1                    redirect.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls -l > out.txt   */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                              /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
```

Figure 3.1: The source of redirect.c in the editor

nano redirect.c – the program does by hand what the shell does for ls -l > out.txt. The child closes the descriptor 1, opens out.txt with the flags O_WRONLY | O_CREAT | O_TRUNC and the permissions 0644, and prints the obtained number on the standard error, which is still the terminal; then it becomes ls. The message is sent deliberately to stderr with fprintf(stderr, ...), because the descriptor 1 is already the file and the message would otherwise be written inside out.txt. The parent only waits and prints a final line. The complete source is given below.

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls -l > out.txt   */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                                   /* descriptor 1
    is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC,
    0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to
    1, as always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

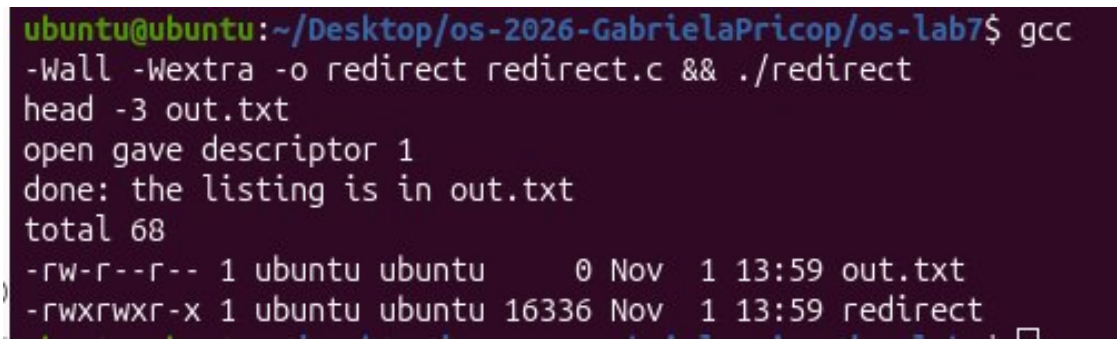


Figure 3.2: The descriptor given by `open` and the content of out.txt

gcc -Wall -Wextra -o redirect redirect.c && ./redirect ; head -3 out.txt – the screen shows only two lines from the program: open gave descriptor 1, written by the child on the standard error, and done: the listing is in out.txt, written by the parent after

waitpid. The listing of ls -l is nowhere on the screen. The command head -3 out.txt then prints the first three lines of the file: total 68, the entry of out.txt itself and the entry of redirect. The size of out.txt in this listing is 0, because ls read the directory immediately after O_TRUNC had emptied the file and before anything was written into it; this detail confirms that the file was created by the child before exec.

Descriptor	**Name**	**Normally**	**To redirect it**
0	standard input	the keyboard	close(0), then open
1	standard output	the terminal	close(1), then open
2	standard error	the terminal	close(2), then open

Table 3: The three standard descriptors and the way each of them is redirected

Table 3 shows that the three redirections differ only by the number that is closed, and that the rule of the smallest free descriptor is what makes the whole mechanism work.

Part 4: A pipe, ls /etc | wc -l

Questions:

- Did ./pipeline and ls /etc | wc -l give the same number?
- After the change, which of the two children never ends, ls or wc?
- Why, in one sentence: what is wc waiting for?

Answers:

- Yes. Both printed 247: first the program written by hand, then the real shell pipeline. The equality of the two numbers proves that the program reproduces exactly what the shell does for ls /etc | wc -l.
- wc. After the writing end p[1] is left open in the parent, ls still ends normally, because it writes everything it has and exits; wc is the one that never ends, so the program hangs and must be stopped with Ctrl-C.
- wc is waiting for the end of file of the pipe, which the kernel reports only when the last writing end has been closed, and the parent still keeps one open, so the read never returns 0.

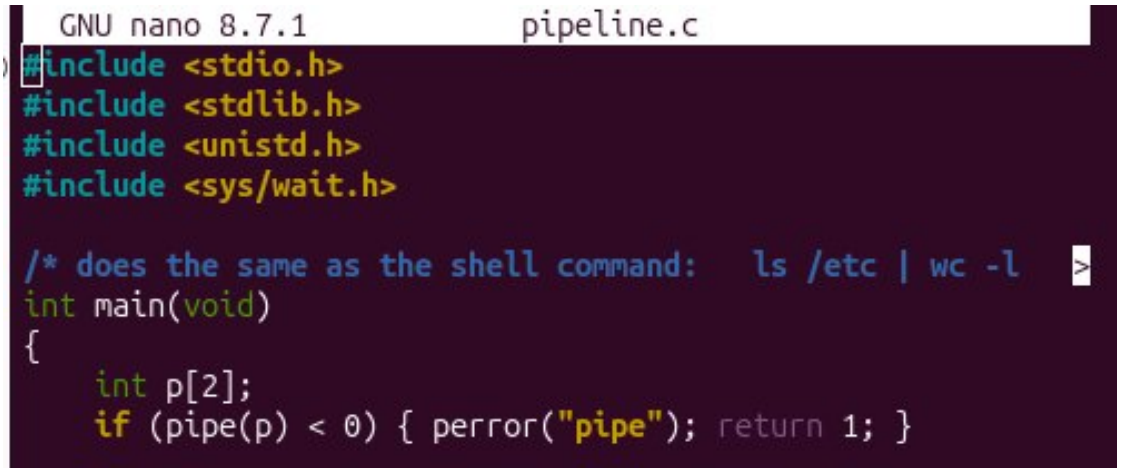


Figure 4.1: The beginning of pipeline.c in the editor

nano pipeline.c – the program creates one pipe with pipe(p), which gives two descriptors: p[0] for reading and p[1] for writing. Then it forks twice. The first child closes the descriptor 1 and calls dup(p[1]), which copies the writing end into the smallest free number, that is 1, so that ls writes into the pipe without knowing it; it then closes both original ends and becomes ls /etc. The second child does the symmetric thing: it closes 0, calls dup(p[0]), which gives 0, closes both ends and becomes wc -l. The parent uses neither end, so it closes both and waits twice, once for every child. The complete source is given below.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls /etc | wc -l   */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                              /* ls: its 1
    becomes p[1] */
        close(1);
        dup(p[1]);                                  /* smallest
    free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                              /* wc: its 0
    becomes p[0] */
        close(0);
```

```
24          dup(p[0]);                                  /* smallest
    free number: 0 */
25          close(p[0]);
26          close(p[1]);
27          execlp("wc", "wc", "-l", (char *)NULL);
28          perror("exec wc");
29          exit(127);
30      }
31
32      close(p[0]);                                    /* the parent
    uses neither end */
33      close(p[1]);
34      wait(NULL);
35      wait(NULL);
36      return 0;
37  }
```

```
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab7$ gcc
-Wall -Wextra -o pipeline pipeline.c && ./pipeline
ls /etc | wc -l
247
247
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab7$
```

Figure 4.2: The program and the real shell print the same number

gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline ; ls /etc | wc -l – the program prints 247 and the shell pipeline prints 247 as well. Neither ls nor wc was modified: ls writes its names into the descriptor 1 and wc counts the lines that arrive on the descriptor 0, exactly as they do on a terminal. The whole connection was made by the parent before exec, with pipe, close and dup.

```
    close(p[0]);                                    /* the par
    /* close(p[1]);*/
    wait(NULL);
    wait(NULL);
    return 0;
}
```

Figure 4.3: The line `close(p[1])` of the parent turned into a comment

/* close(p[1]); */ – the program was broken on purpose: in the parent, the line that closes the writing end was turned into a comment, while close(p[0]) was left in place. The pipe now has three writing ends in total, the one of the child ls and the one that the parent keeps, and the child closes its own when it ends.

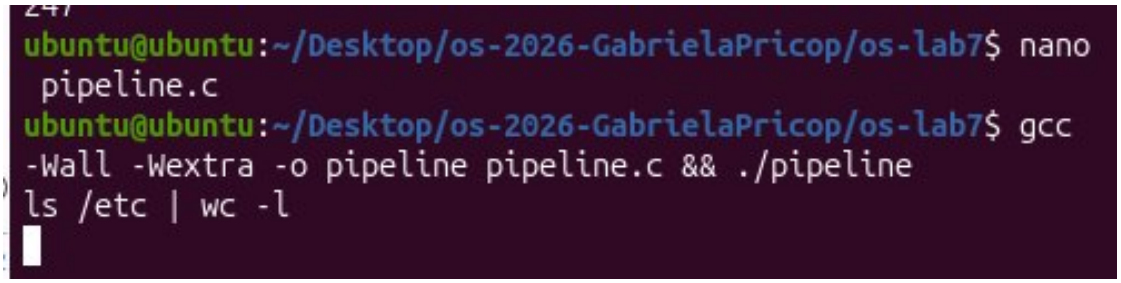


Figure 4.4: After the change the program never ends and must be stopped with Ctrl-C

./pipeline (after the change) – the program no longer prints anything and does not return the prompt. ls ends normally, but wc keeps reading: the kernel counts how many processes still hold the writing end open, and it reports the end of file only when that number falls to zero. The parent holds one open and is itself blocked in wait, so the two processes wait for each other and the program hangs until Ctrl-C is pressed. The two situations are compared in Table 4.

State of the parent	**Writing ends open**	**Result**	**Reason**
close(p[1]) present	only that of ls, then none	247, program ends	wc receives the end of file
close(p[1]) commented	that of the parent stays open	blocked, Ctrl-C	wc never receives the end of file

Table 4: The effect of the unclosed writing end of the pipe

Table 4 shows that the correctness of a pipeline depends on closing every end that is not used, and that one forgotten descriptor in a process that does not even use the pipe is enough to block the whole chain.

Conclusion

The laboratory work showed, with four short programs, that the shell is not a special component of the system but an ordinary process that uses a small number of system calls. In the first part, run.c reproduced the birth of a process: fork produced a copy that differs from its parent only by the returned value, execlp replaced the program of the child, and waitpid gave the parent the status of the child. The same program launched ls, then date, with the change of a single line and with the status 0 in both cases, and with nosuchprog it showed the opposite situation: execlp returned, perror printed exec: No such file or directory and the status became 127, which proves that a successful exec never returns.

In the second part, the single line sleep(20) added before waitpid created a zombie on purpose: ps showed [ls] <defunct> in state Z, with the PPID of the process ./zombie, and after the 20 seconds the call to waitpid removed it from the process table. In the third part, redirect.c performed the redirection ls -l > out.txt by hand: close(1) followed by open returned the descriptor 1, because open always gives the smallest free number, and the unmodified ls wrote its listing into the file. In the fourth part, pipeline.c built ls /etc | wc -l with one pipe, two children and dup, and printed 247, the same number as the real shell; leaving the writing end open in the parent blocked wc for ever, because the end of file of a pipe appears only when the last writing end is closed.

Taken together, the four parts gave a practical understanding of the life cycle of a process, of the meaning of the exit status, of the rule of the smallest free descriptor and of the discipline of closing the ends of a pipe. All the objectives of the laboratory work were achieved, and the programs behave exactly as the corresponding commands of the shell.

References

1. *Gabriela Pricop. os-2026-GabrielaPricop: laboratory works of the Operating Systems course, GitHub repository*
   `https://github.com/gabriela-pricop/os-2026-GabrielaPricop`
2. *fork(2) – create a child process, Linux manual page*
   `https://man7.org/linux/man-pages/man2/fork.2.html`
3. *pipe(2) – create pipe, and the end-of-file condition of a pipe, Linux manual page*
   `https://man7.org/linux/man-pages/man2/pipe.2.html`
4. *dup(2) – duplicate a file descriptor, and the rule of the lowest-numbered unused descriptor, Linux manual page*
   `https://man7.org/linux/man-pages/man2/dup.2.html`